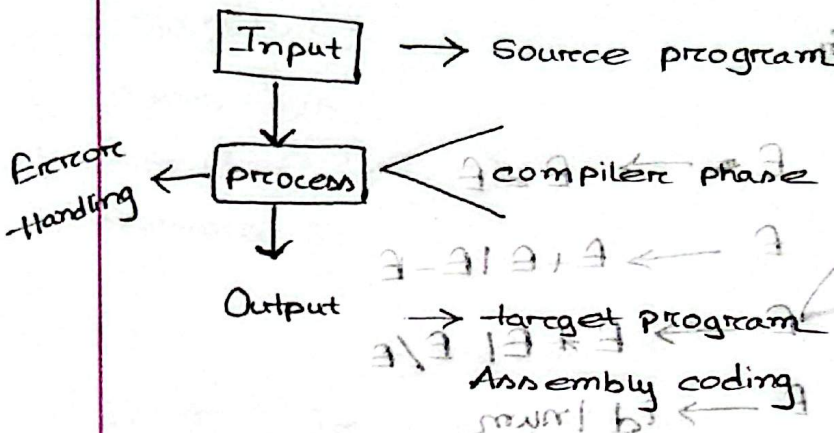
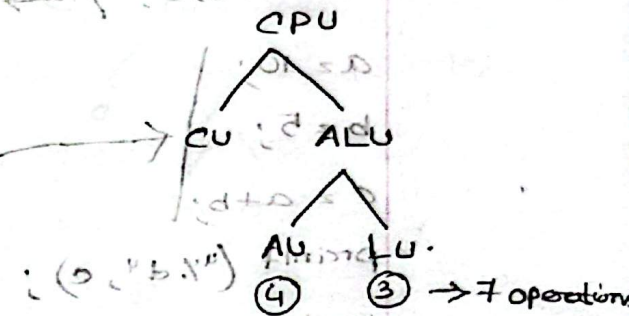
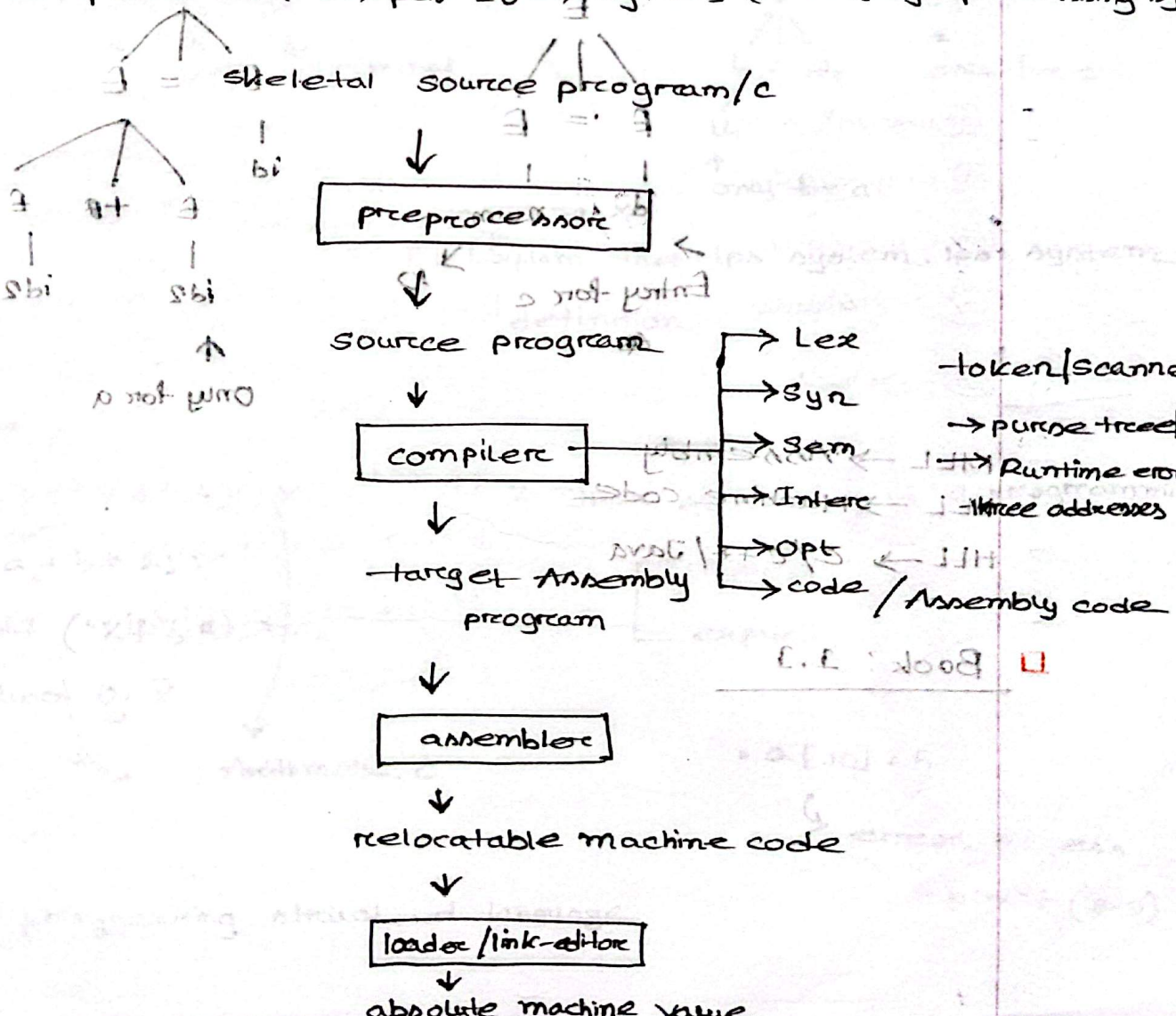
Compiler Introduction





☐ Software tools – ৭/৫ বিস্তারিত, (Book page-18) ☐ Book: 1.2 Analysis

☐ compiler] book → pdf 20 → fig: 1.3 (A language processing syste



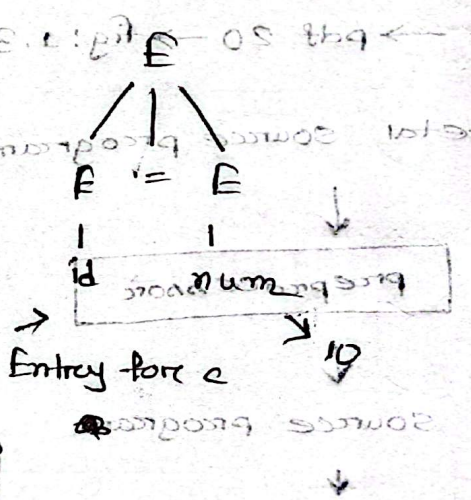
```
int main() {
int a,b,c; /→ CFG
a=10;
b=5;
c=a+b;
printf("%d", c);
return 0;
}
```

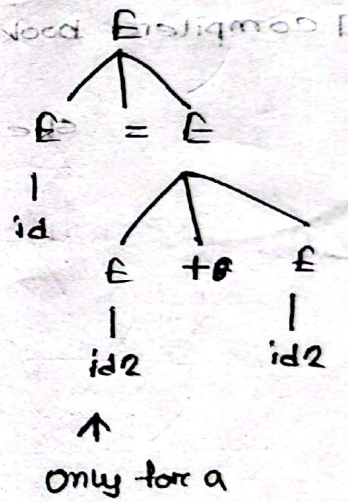
E → E=E

E → E+E | E-E

E → E*E | E/E

E → id | num





HLL → C/C++/Java

MLL → Assembly

LLL → Machine code

Book : 1.1

```
int main () {
int a, b, c;
c = a + b/5;
printf ("%d", c)
return 0;
}
```

D → T L ;

T → int / long / float / double

L implies L , id

(L : variable, id : terminal)

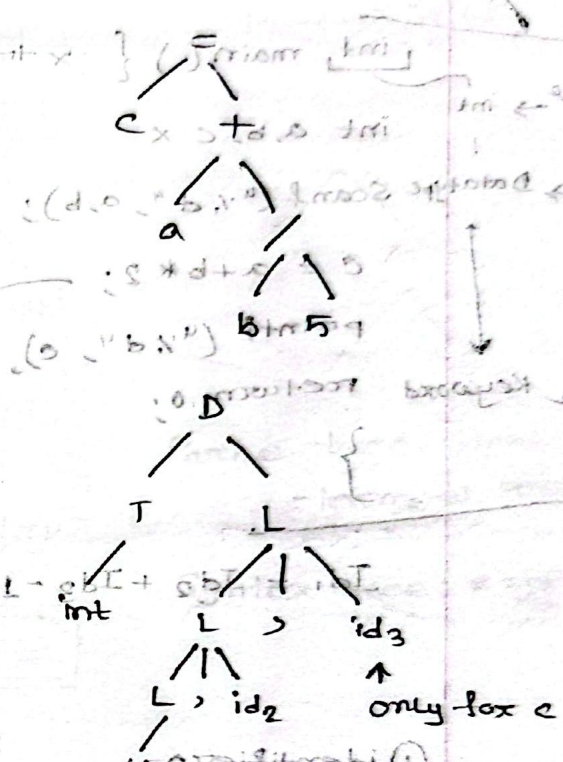


fig: 1.9

```
int main () {
int a, b; x
float c; x
a = 5; ✓
scanf ("%d", b); x
c = a + b * 2; ✓
printf ("%lf", d); x
return 0; x
} x
```

->translation-

Arithmetic.c

Explain the lps system. lps system definition.

2 types of casting in c programming.

- implicit
- explicit

- a[-10] = 5

↳ error access bound (0-9)

C programming structured language

Tokenisation

Lexical analysis

Lexeme → int

Pattern → Datatype

Ctaken Keyword

```
int main() {
int a,b,c x
Scanf("%d", a,b); x
c = a+b*2;
printf("%d", c);
return 0;
}
```

$Id_1 = Id_2 + Id_3 * 1$

(i) identifiers
(ii) Keywords
(iii) Operators
(iv) constants
(v) Symbols

(7)

$t_1 = a$
$t_2 = b$
$t_3 = a+b$
$t_4 = t_3 \times 2$
$c = t_4$

Optimization

$t_1 = a+b$
$c = t_1 * 2$

(4)

$E \to E * E$
$E \to E + E \mid E * E$
$E \to E * E \mid E * E$
$E \to id \mid num$

Ambigeous

$E \to E = E$
$E \to E{+}{+} \mid E{-}{-} \mid T$
$T \to T * F \mid T / F \mid F$
$F \to id \mid num$

Unambiguous

$E \to E = E$
$E \to TE'$
$E' \to +TE \mid -TE' \mid \epsilon$
$T' \to *FT' \mid /FT' \mid \epsilon$
$F \to id \mid num$

Unambiguous + left recursion elementation

Lexical Analyzer

scanner

$id_1 = 5$

$id_3 = id_1 + id_2 * 1$

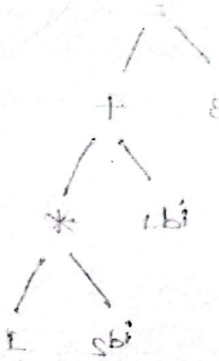
t_1

t_2

Syntax Analyzer

Parser

Parsing table based on CFG



Parse tree: Start থেকে terminal পর্যন্ত

Syntax tree: শুধু terminal

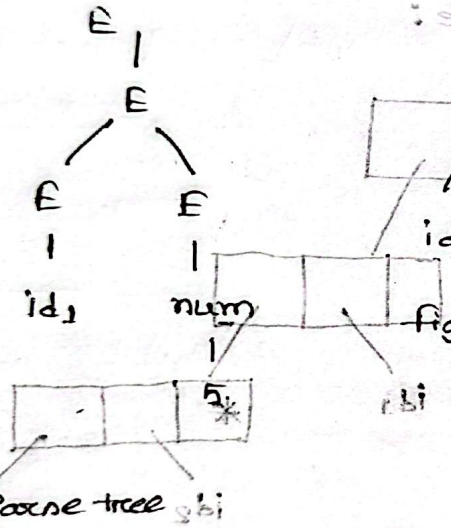


fig: Parse tree

entry for a

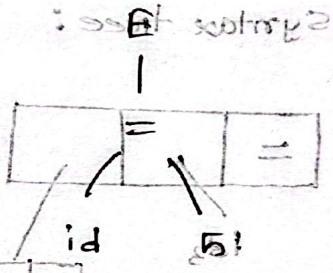
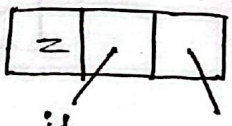


fig: Syntax tree



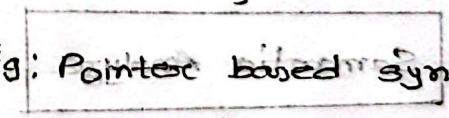
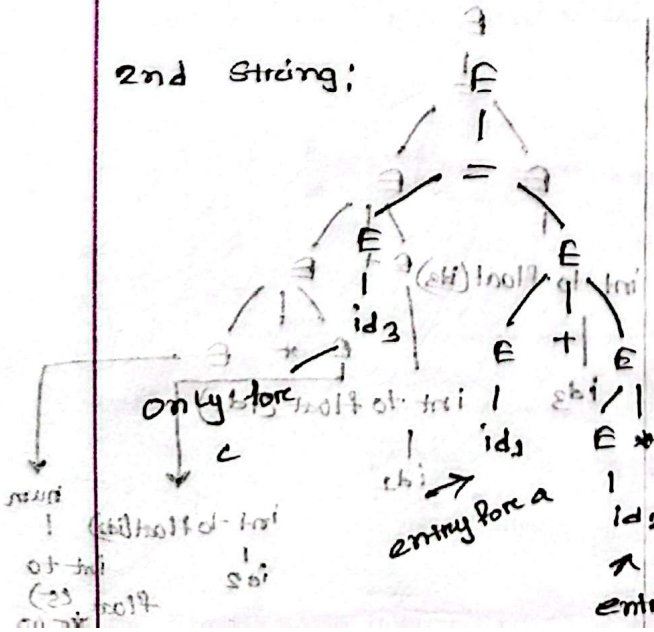


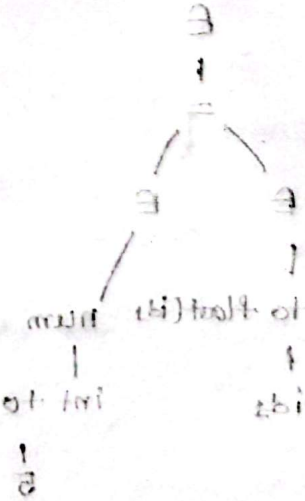
fig: Pointer based syntax tree

2nd String:

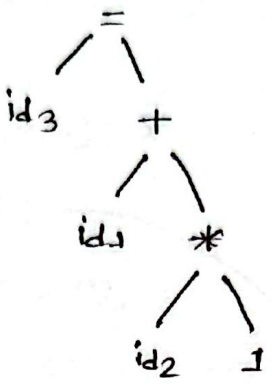


entry for a

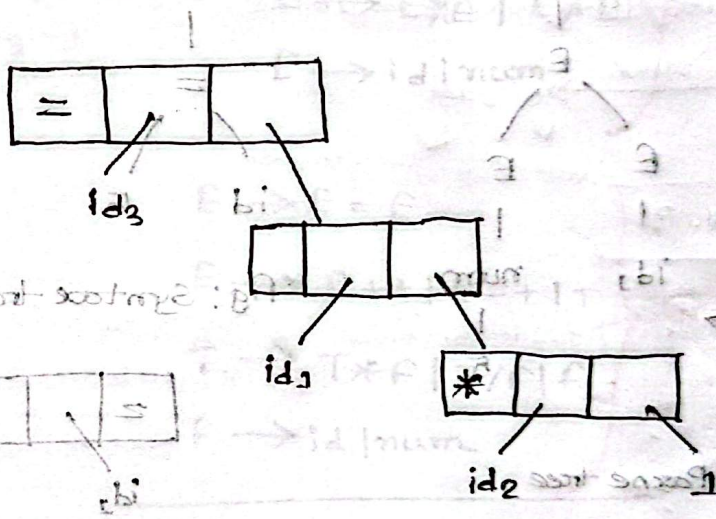
entry for b



- Abstract Syntax tree:

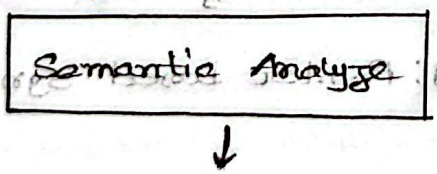


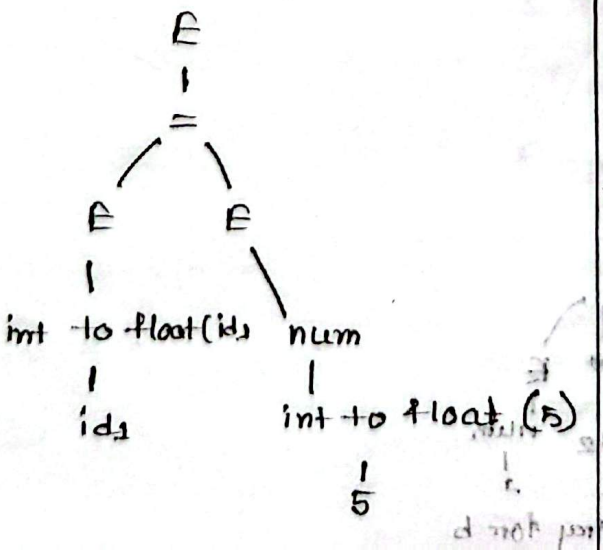
- Pointer based syntax tree:

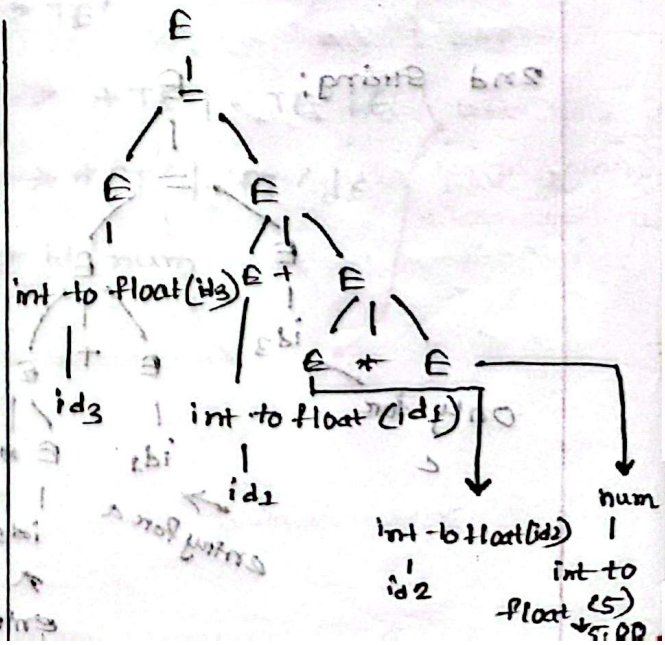




Static checker
Runtime environment (memory + typecast)







↓

Intermidiate code generator

$t_1 = \text{int to float}(5)$

$id_1 = t_1$

$t_1 = id_1 + id_2$

$t_2 = \text{int to float}(1)$

$t_3 = t_1 * t_2$

$id_3 = t_3$

↓

code optimization

$id_1 = 50$

$t_1 = id_1 + id_2$

$id_3 = t_2 * 10$

↓

code generator (tools)

↓

MOVF id_1, #5.0

MOVF R1, id_1 → MOVF R2, Id2

ADDF R1, id2

MULF R1, #1.0

MOVF id_3, R1

Destination ↙

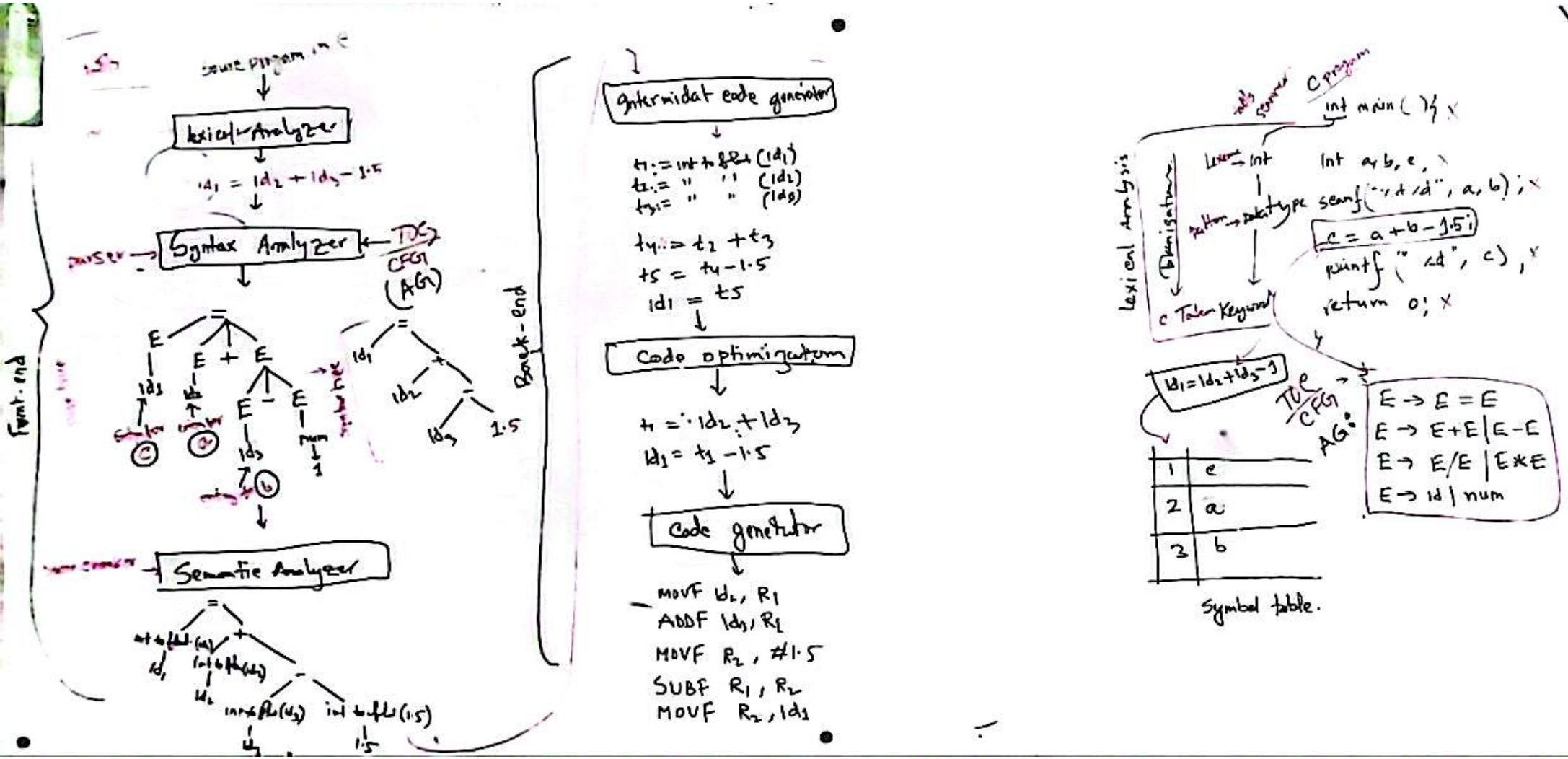
[সরাসরি value দিলে # দিতে হবে, নাহলে address ধরবে]

☐ Difference between Syntax Directed Definition and Syntax directed translation (semantic Rules)

⇒ Syntax Directed Defn	Syntax Directed translation
$E \rightarrow E + T$	$E.val = E_1.val + T.val$
$E \rightarrow E - T / T$	
$T \rightarrow T * F \mid T / F \mid F$	
$F \rightarrow id \mid num$	

Maishaa

CamScanner


source program in C
Lexical Analyzer
id1 = id2 + id3 - 1.5
Syntax Analyzer
TDS
CFG
(AG)
parser
Front-end
Back-end
Semantic Analyzer
Intermidiat code generator
t1 := int to float (id1)
t2 := " " (id2)
t3 := " " (id3)
t4 := t2 + t3
t5 = t4 - 1.5
id1 = t5
Code optimization
t1 := id2 + id3
id1 = t1 - 1.5
Code generator
MOVF id2, R1
ADDF id3, R1
MOVF R2, #1.5
SUBF R1, R2
MOVF R2, id1

C program
int main( ){
Lexical analysis
Tokenization
Lexeme → int
int a, b, c;
pattern → prototype
scanf("%d%d", a, b);
c = a + b - 1.5;
printf("%d", c);
return 0;
c Token Keyword
id1 = id2 + id3 - 1
TDS / CFG / AG
E → E = E
E → E + E | E - E
E → E / E | E * E
E → id | num
1 e
2 a
3 b
Symbol table.

```
int main(){ X
float c; X
int a, b; X
b=10; ✓
scanf("%d",&a); X
c = a*b-1; ✓
printf("%f", c); X
return 0; X
} X
```

for(i=1; i<=c; i++){ ✓
b = a+i; ✓
}

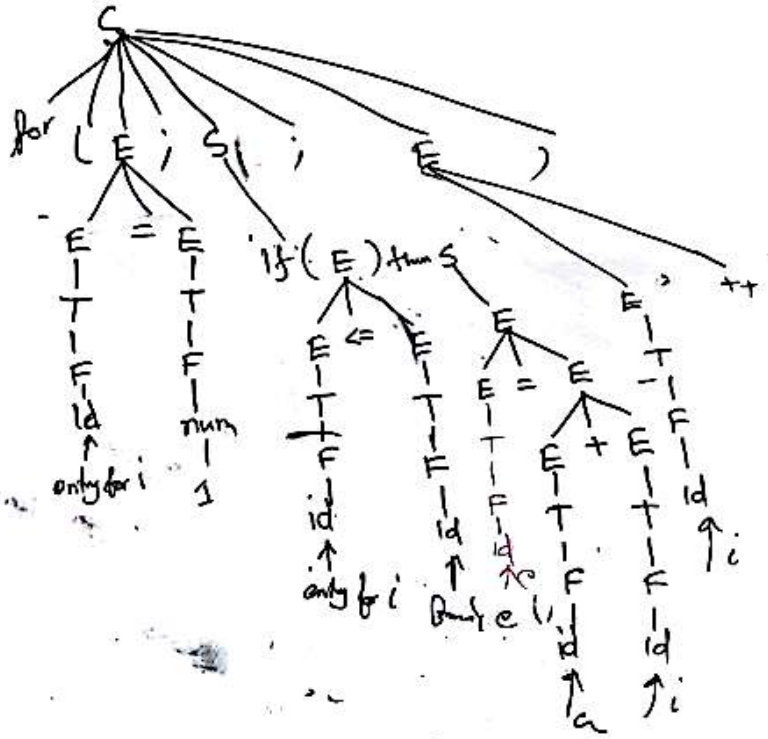
S → E | for(E;S;E) | if (E) then S

E → E:=E | E<=E
E → E+T | E-T | T | E++
T → T*F | T/F | F
F → id | num

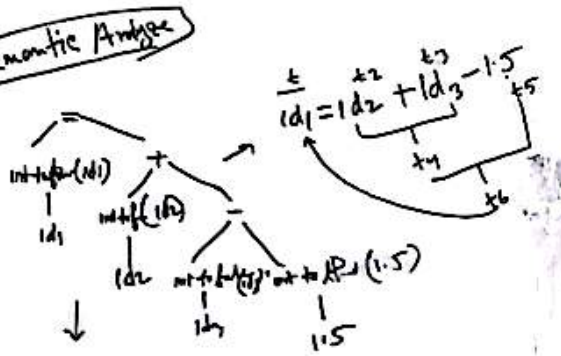
CFG

Syntax directed definition

E → E' | TE'
E' → =EE' | +TE' | -TE' | ∈
T → FT'
T' → *FT' | /FT' | ∈
F → id | num



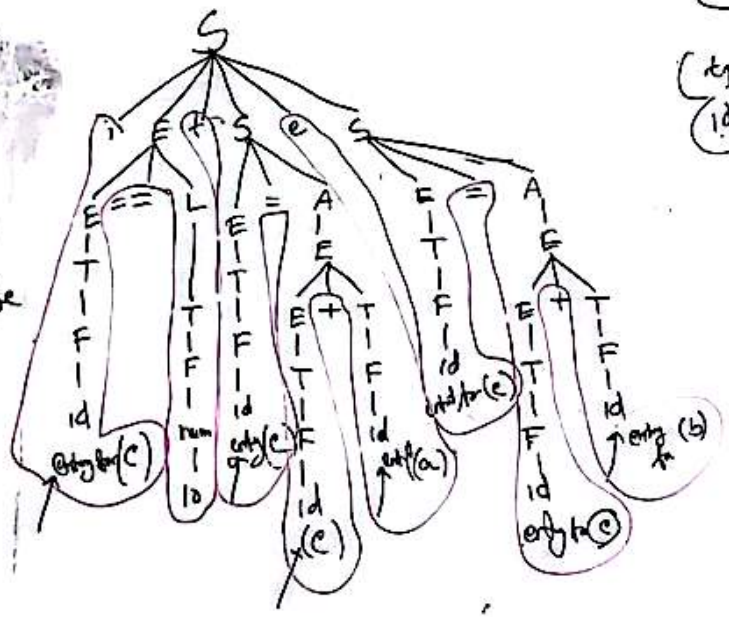
Semantic Analysis



Intermediate code generation

$t_1 = \text{int to float}(id_1)$ X
$t_2 = \text{"} \quad (id_2)$
$t_3 = \text{"} \quad (id_3)$
$t_4 = t_2 + t_3$
$t_5 = \text{int to float}(1.5)$
$t_6 = t_4 - t_5$
$id_1 = t_6$

Code optimization



there Address code

Code optimization

$t_1 = id_2 + id_3$) R1
$id_1 = t_1 - 1.50$

```
MOVF  id2, R1      MOV id3, R2
ADDF  id3, R1
MOVF  #1.5, R2
SUBF  R1, R2
MOVF  R1, id1
```

```
int main(){
float a,b;
int c;
scanf("%f%f", &a, &b);
c = a+b/5;
if(c==10){
    c = c+a;
}
else {c = c+b;
}
printf("%d", c);
return 0;
}
```

a b

S → E = A | iE+SeS | E
A → E
E → E+T | E-T | T | E==L
T → T*F | T/F | F
F → id | num
L → T

e-conditional Statement (if-else)

① if(Exp){
    Stmt;
}

② if(Exp){
    stmt;
}
else{
    stmts;
}

③ if(Exp){
    stmt;
}
else if(Exp){
    stmt;
}
⋮
else{
    stmt
}

} S

Syntax definition (CFG)

S → if (E) then S
S → S else S
S → S elseif (E) then S
S → ∧
E → E == E | E != E | E <= E | E >= E
E → E || E | E && E | !E

S → iE+S | SeS | SeiE+S | a
E → b

```
if(cond){
  ...
}
else if (cond...){
  ....
}
else if (cond...){
  ....
}
else
```

S → S else S
→ S elseif (E) then S else S
→ S elseif (E) then S elseif (E) then S else S
→ if (E) then S elseif (E) then S elseif (E) then S else S

Basic compiler construction tools

1. Scanner generator
2. Parser
3. Syntax directed translation engin
4. Automatic Code generator
5. Data flow engin.

Source program analysis tools

1. Structure editor.
2. pretty printer
3. Static checker.
4. Interpreter

Syntax directed definition

Syntax directed translation (Semantic Rules)

$E \rightarrow E+T \mid E-T \mid T$	E.val = E1.val + T.val (Attributes) | E.val - T.val
$T \rightarrow id \mid num$	T.val = id.entry | num.lexval / num.val

A simple One-Pass Compiler

- Precedence of Operator

Ex: 2.1 - 2.8

Top-down (left most derivation)

Bottom up (right " ")

=> scan করে যেটা পাবো lexeme

```
int a = 10;
```

Lexeme	Pattern	Token
int	data type	keyword
a	variable name	identification
=	assignment	operators
10	number	constent
;	symbol	symbol

id

	a

ToC.

→ RE

→ CFG

→ DFA

→ NFA

→ ϵ - NFA

Big sigma

$\Sigma = \{0, 1\}$

```
int main(){
int a = 10, b = 5, c
   c = a+b;
printf("%d", c);
return 0;
}
```

lexeme	pattern	token
int	data type	keyword
main	function name	identifier

- Identifier rules → start with underscore (_) or letter

- RE :

⇒ ('_'/L) ('_'/L/D)*

⇒ ('_' | 'a-z' | 'A-Z') ('_' | 'a-z' | 'A-Z' | '0-9')*

- CFG :

I → _A | LA

A → _A | LA | DA

L → 'a-z' | 'A-Z' / L → 'a' | ... | 'z' | 'A' | ... | 'Z'

D → '0' | ... | '9'

Production → 68

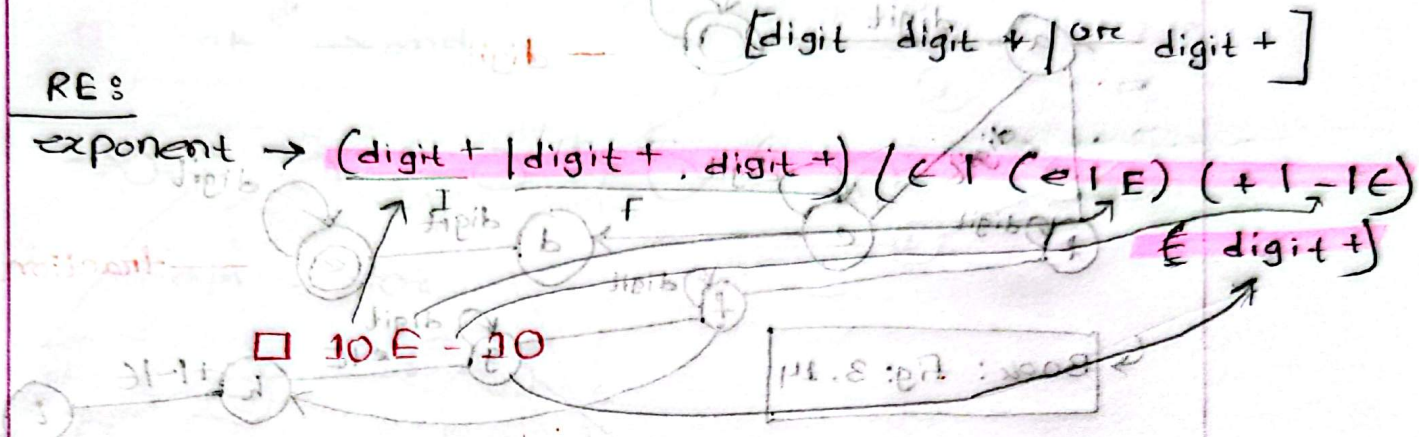
- Book Page - 94 (Difference)
- " " - 96
- Example : 3.4

Fraction →

digit . digit + → fraction part

RE:

- exponent → (digit + | digit + . digit +) (ε | (e | E) (+ | - | ε) digit +)



- 10.E-10

CFG:

N → IP | FP

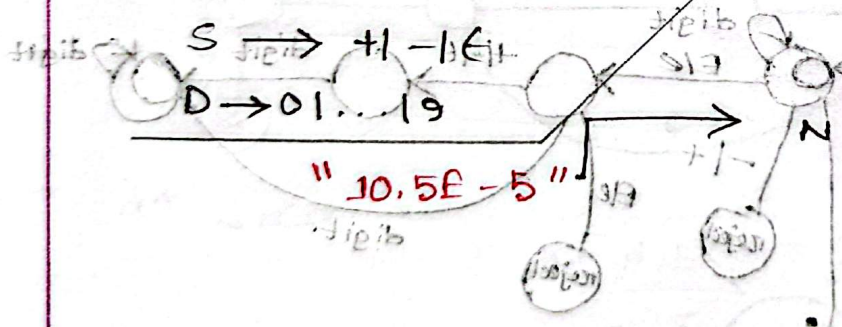
I → digit I | digit

F → d I . I

P → eSI | 'E'SI | ε

S → + | - | ε

D → 0 | 1 | ... | 9



"10.5E-5"

Leftmost

N → FP

→ I . IP

→ 1D . I PI

→ DD . IP

→ 1D . IP

→ 10 . IP

→ 10 . DP

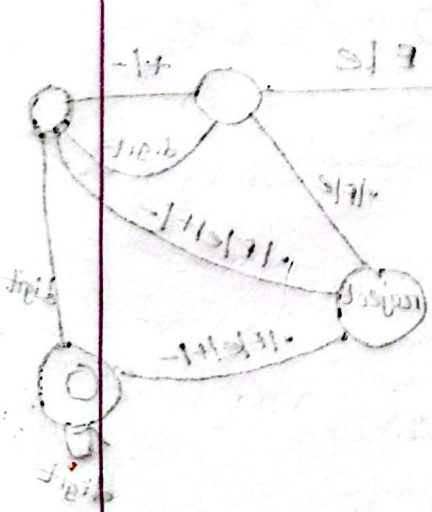
→ 10 . 5P

→ 10 . 5E SI

→ 10 . 5E - I

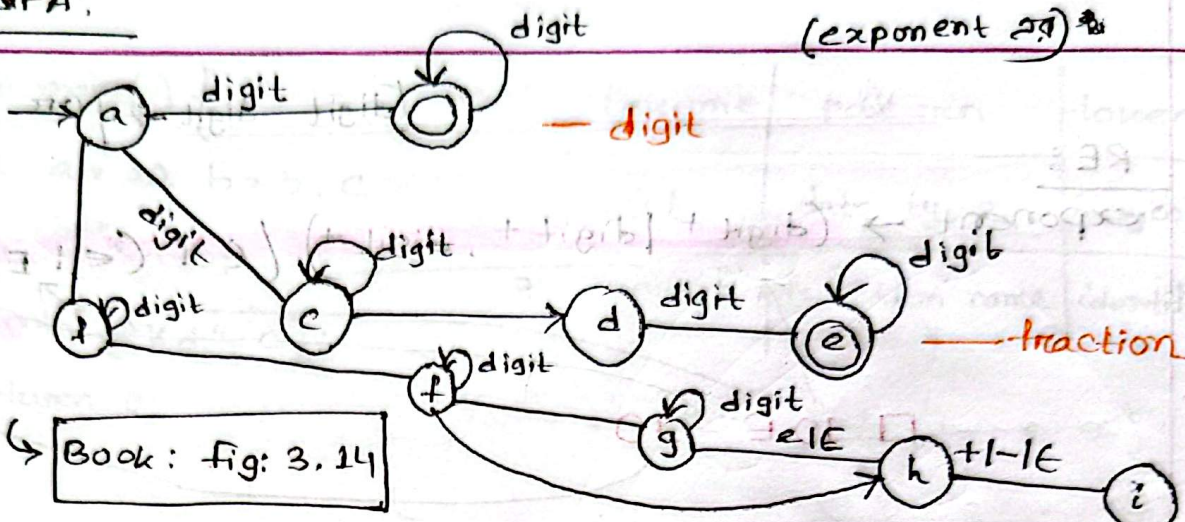
→ 10 . 5E - D

→ 10 . 5E - 5



• NFA:

(exponent এর)


digit
digit
digit

Identifier

CFG

$I \to _A \mid LA$

$A \to _A \mid LA \mid DA \mid \epsilon$

$L \to a \mid \dots \mid z \mid A \mid \dots \mid Z$

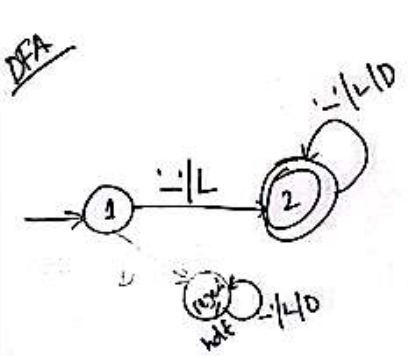
$D \to 0 \mid \dots \mid 9$

RE:

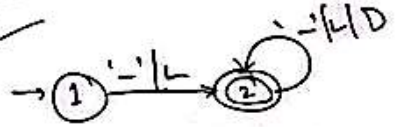
$(_ \mid a\text{-}z \mid A\text{-}Z)\ (_ \mid a\text{-}z \mid A\text{-}Z \mid 0\text{-}9)^*$

$(_ \mid L)\ (_ \mid L \mid D)^*$

DFA



NFA



Number

Integer:

$\frac{DD^*}{D^+}\ (\epsilon \mid +\mid -)\ (0\text{-}9)\ (0\text{-}9)^*$

$(0\text{-}9)^+$

Fraction:

$digits^+ . digits^+$

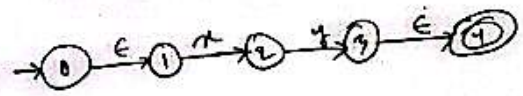
Exponential:

$digits^+ . digits^+ (e \mid E)\ (\epsilon \mid + \mid -)\ digits^+$

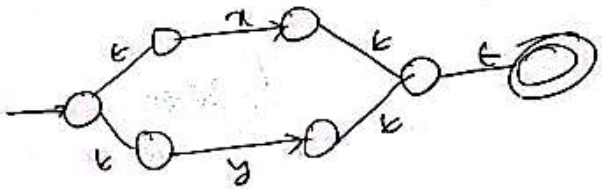
Number:

$(digits^+ \mid digits^+ . digits^+)\ (\epsilon \mid [(e \mid E)\ (\epsilon \mid + \mid -)\ digits^+])$

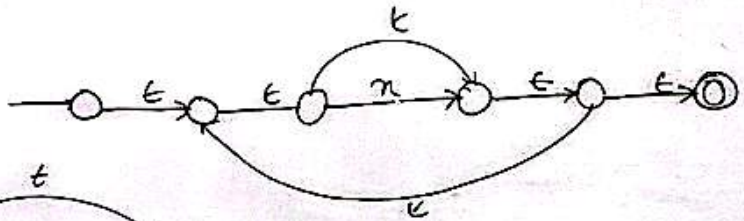
xy:



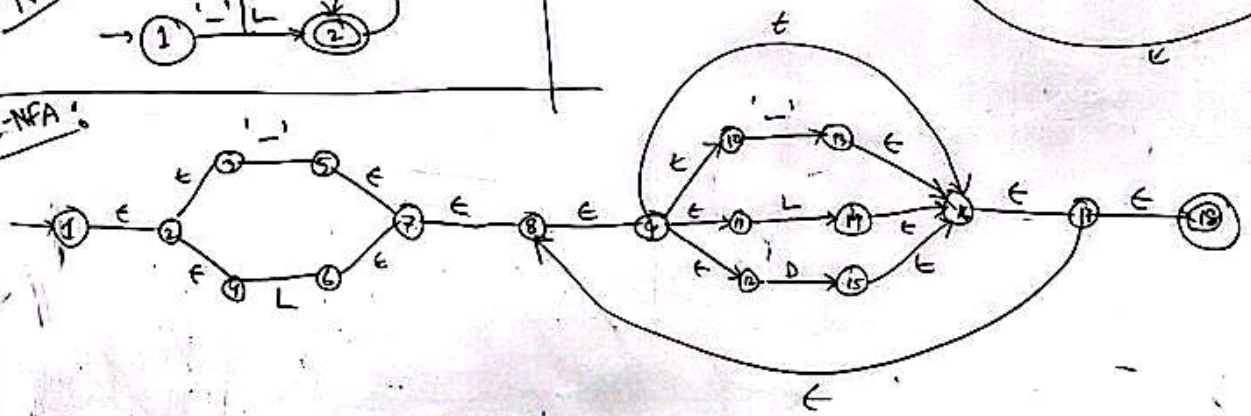
x|y:



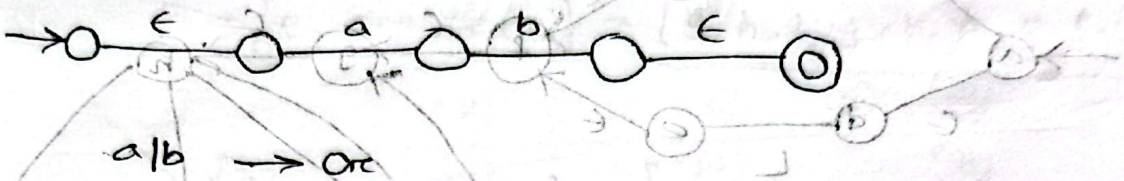
x*:



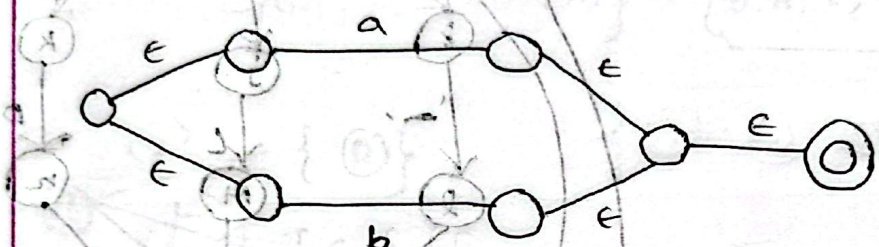
ε-NFA:



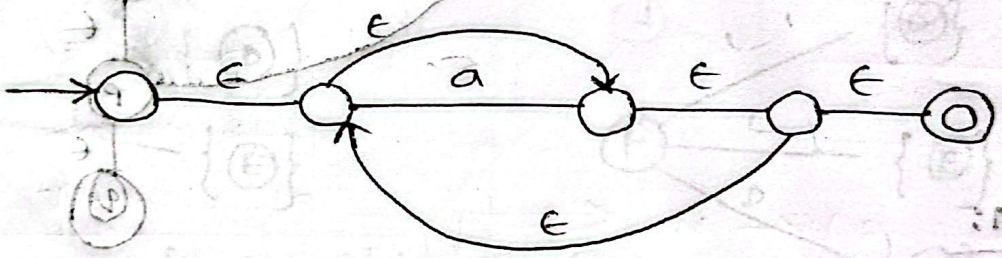
ab → and



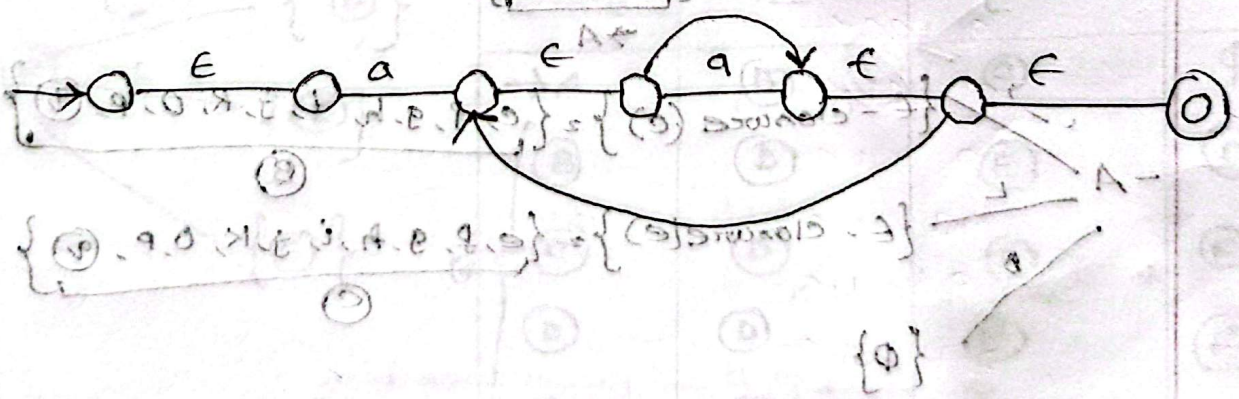
a|b → or



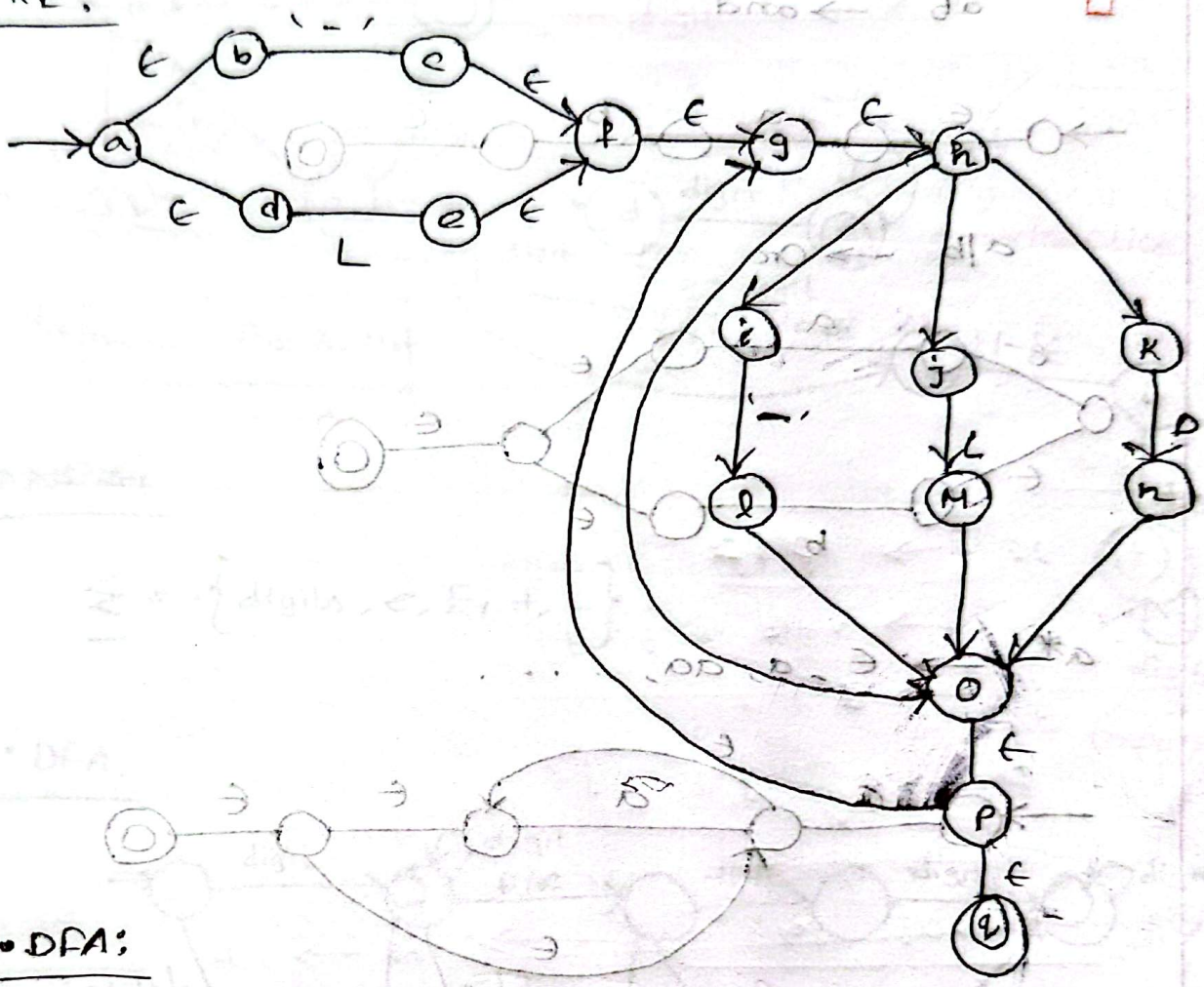
a* → ε, a, aa, ...



a+ = aa*



• RE:



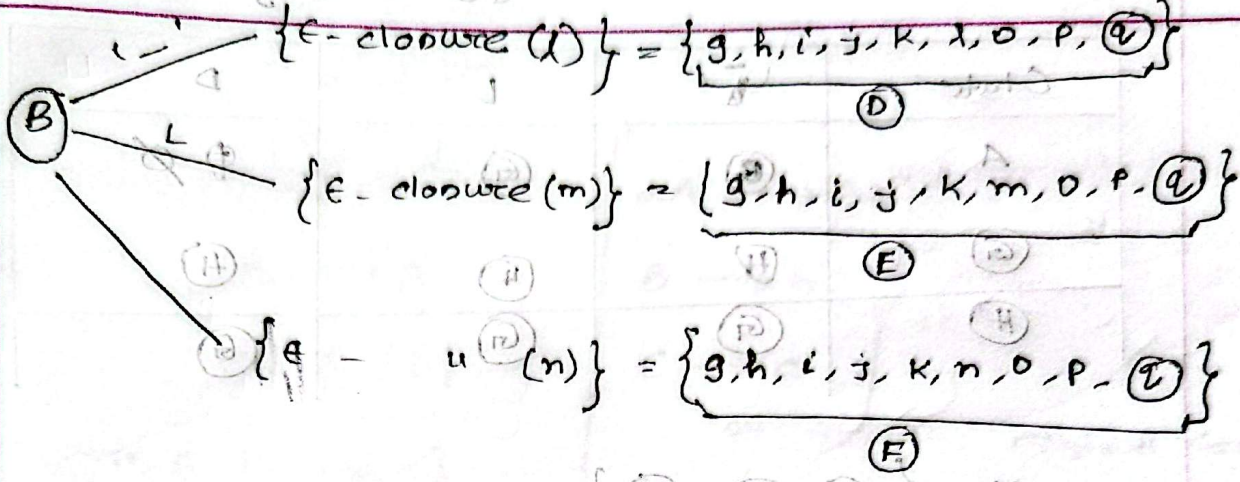
• DFA:

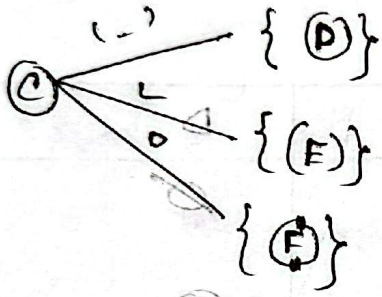
ε - closure (→a) = {→a, b, d}
→A

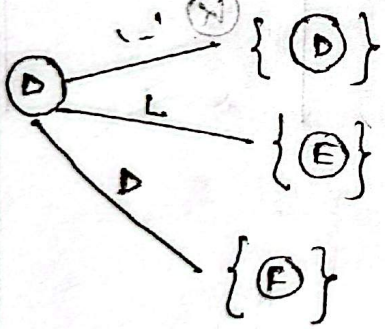
A —L→ {ε - closure (c)} = {c, f, g, h, l, j, K, O, P, q}
B

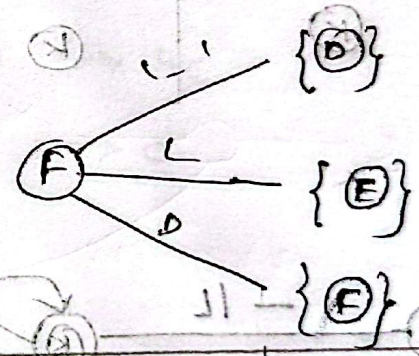
A —L→ {ε - closure (e)} = {e, f, g, h, i, j, K, O, P, q}
C

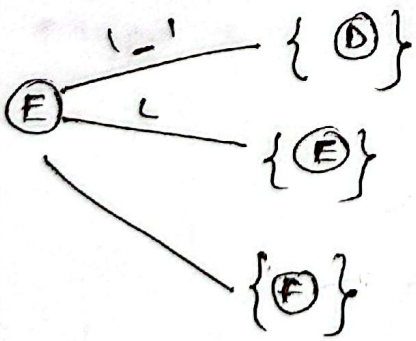
A —D→ {Φ}











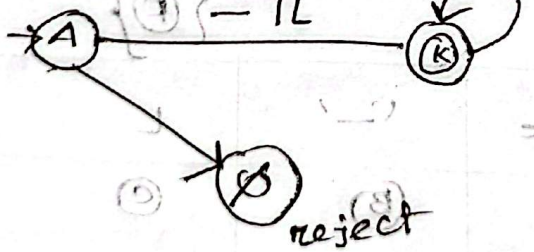
State	'_'	L	D
→A	B	C	ϕ
B	D	E	F
C	D	E	F
D	D	E	F
E	D	E	F
F	D	E	F

$\{D, E, F\} = H$

State	'_'	L	D
A	G	G	ϕ
G	H	G	H
H	G	G	G

$K = \{G, H\}$

State	'_'	L	D
A	K	K	ϕ
K	K	K	K



'_' / L | D

* Type 0 = UG

Production head — $\alpha \rightarrow \beta$ — production body

$\alpha \neq \phi$

UG (unrestricted grammar)

$AA \rightarrow A$

$A \rightarrow a$

Types of Grammar :

- type 1 = CSG

$$\alpha \longrightarrow \beta$$

$$|\alpha| \leq |\beta|$$

$$\alpha, \beta \in \{V, T\}$$

$$aA \longrightarrow Bb$$

$$B \longrightarrow bC$$

$$C \longrightarrow c$$

CFG

Context Sensitive

- type 2 = CFG

$$\alpha \longrightarrow \beta$$

$$|\alpha| = 1$$

$$\alpha \in \{V\}$$

$$\beta \in \{V, T\}$$

$$A \longrightarrow Bb$$

$$B \longrightarrow bC$$

$$C \longrightarrow c$$

- type 3 =

Regular

Regular Grammar

Linear "

CFG

$$A \longrightarrow AaAB \mid a$$

$$B \longrightarrow b$$

$$S \longrightarrow AB$$

$$A \longrightarrow a$$

$$B \longrightarrow b$$

$$S \longrightarrow aA \mid bB$$

$$A \longrightarrow a$$

$$B \longrightarrow b$$

LG

RLG

CFG

$S \rightarrow aAB$

$A \rightarrow bB$

$B \rightarrow b$

$C \rightarrow dB \mid C$

X

↳ Useless

$T = \{a, b, e, d\}$

Maisha_5AF

RE:

$$\left(\underbrace{digits^+}_{I} \;\middle|\; \underbrace{digits^+ . digits^+}_{F}\right)\left(\epsilon \;\middle|\; \underbrace{\left((E|e)\underbrace{(+|-|\epsilon)}_{S} digits^+\right)}_{P}\right)$$

CFG:

$Num \rightarrow IP \mid FP$

$\rightarrow ESI \mid eSI \mid \epsilon$

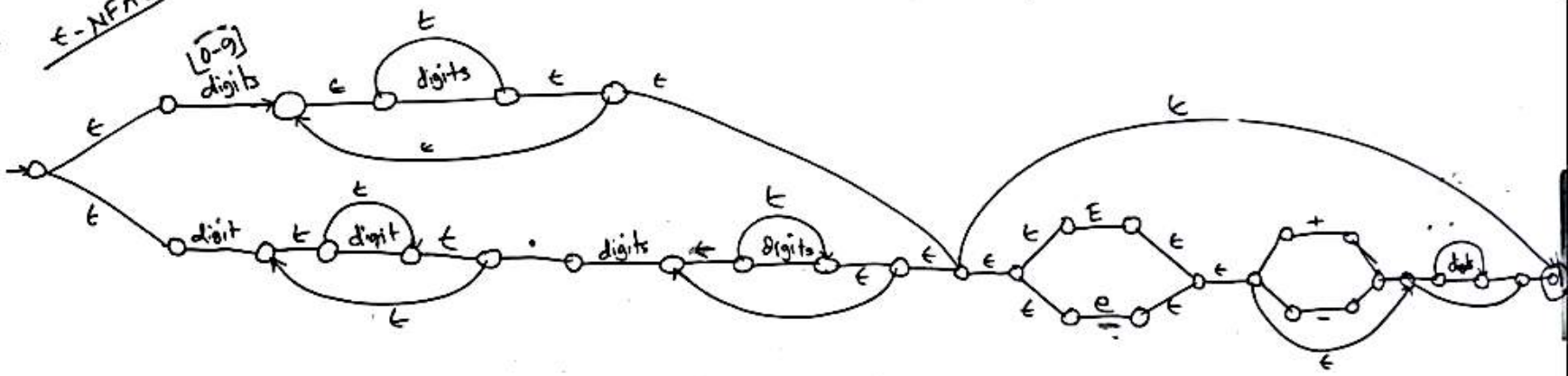
$F \rightarrow I.I$

$I \rightarrow DI \mid D$

$S \rightarrow + \mid - \mid \epsilon$

$D \rightarrow 0 \mid \ldots \mid 9$

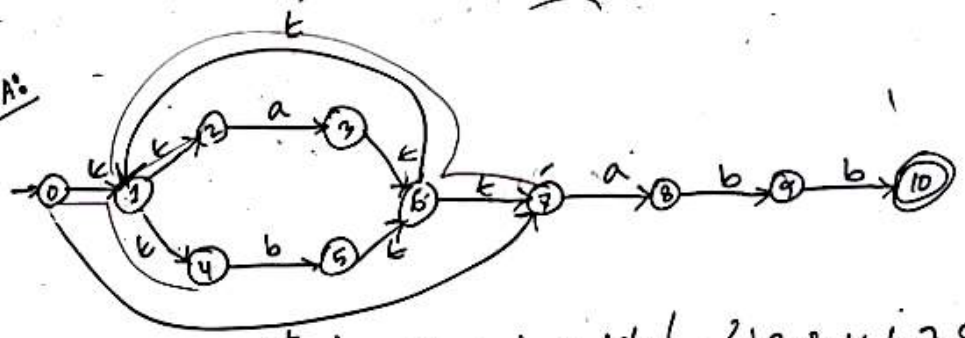
ϵ-NFA:



$\Sigma = \{a, b\}$

RE: $(a|b)^* a b b$

ε-NFA:



State	a	b
→ A	B	C
B	B	D
C	B	C
D	B	E
(E)	B	C

$\varepsilon\text{-closure}\{0\} = \{0, 1, 2, 4, 7\}$ → A

a → $\{\varepsilon\text{-closure}\{3\}, \varepsilon\text{-closure}\{8\}\} = \{1, 2, 3, 4, 6, 7, 8\} = B$

b → $\{\varepsilon\text{-closure}\{5\}\} = \{1, 2, 4, 5, 6, 7\} = C$

B:

a → $\{\varepsilon\text{-closure}\{3\}, \{8\}\} = B$

b → $\{\varepsilon\text{-closure}\{5\}, \{9\}\} = \{1, 2, 4, 5, 6, 7, 9\} = D$

C:

a → $\{\varepsilon\text{-closure}\{3\}, \{8\}\} = B$

b → $\{\varepsilon\text{-closure}\{5\}\} = C$

D:

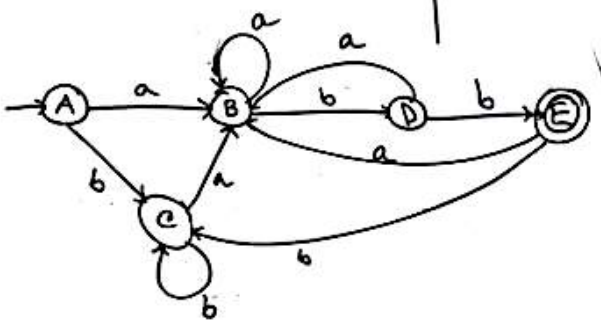
a → $\{\varepsilon\text{-closure}\{3\}, \{8\}\} = B$

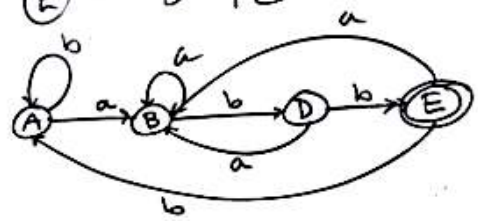
b → $\{\varepsilon\text{-closure}\{5\}, \{10\}\} = \{1, 2, 4, 5, 6, 7, 10\}$ (E)

E:

a → $\{\varepsilon\text{-closure}\{3\}, \{8\}\} = B$

b → $\{\varepsilon\text{-closure}\{5\}\} = C$





Final DFA.

$\Sigma = \{0, 1\}$

L = All String start with 1, and with 01

RE = $1(0|1)^* 01\#$

ϵ-closure $\{a\} = \{a\}$ — A

A → 0 → $\{\emptyset\}$

A → 1 → ϵ-closure $\{b\} = \{b, c, d, f, i\}$ — B

B → 0 → ϵ-closure $\{e\}, \{j\} = \{c, d, e, f, h, i, j\}$ — C

B → 1 → ϵ-closure $\{g\} = \{c, d, f, g, h, i\}$ — D

C → 0 → ϵ-closure $\{e\}, \{j\} = C$

C → 1 → ϵ-closure $\{g\}, \{k\} = \{c, d, f, g, h, i, k\}$ — E

D → 0 → ϵ-closure $\{e\}, \{j\} = C$

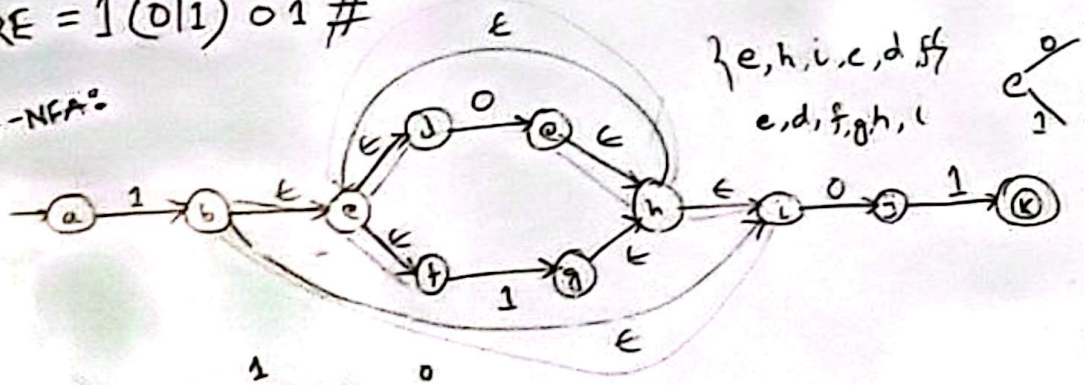
D → 1 → ϵ-closure $\{g\} = D$

E → 0 → ϵ-closure $\{e\}, \{j\} = C$

E → 1 → ϵ-closure $\{g\} = D$

	0	1
→A	$\emptyset$	B
B	C	D
C	C	E
D	C	D
E	C	D

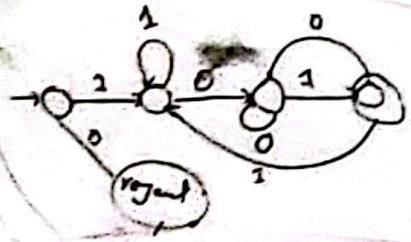
ϵ-NFA:

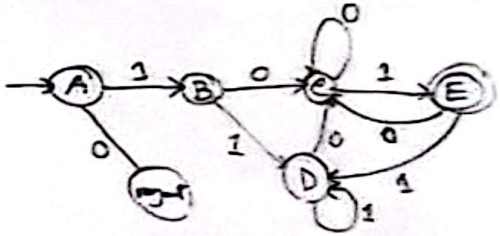


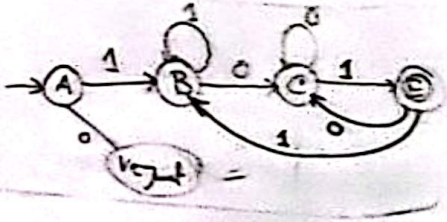
{e, h, i, c, d, f}

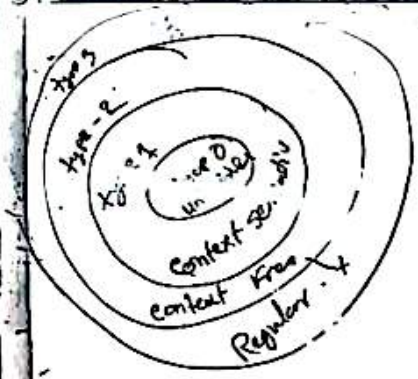
e, d, f, g, h, i

DFA:









type 0 :- Unrestricted grammar (UG)

$\alpha \rightarrow \beta$

$|\alpha, \beta| \neq \phi$

$AA \rightarrow /aA$
$A \rightarrow aB$
$B \rightarrow b$

Type-1 → ~~Context~~ sensativ grmm (CSG)

$\alpha \rightarrow \beta$

$|\alpha| \leq |\beta|$

$\alpha, \beta \in \{V, T\}$

$aS \rightarrow aA$,
$A \rightarrow aB$
$aB \rightarrow b$

type-2 : Context Free Grammar (CFG)

$\alpha \rightarrow \beta$

$|\alpha| = 1$

$\alpha \in \{V\}, \beta \in \{V, T\}$

type 3 : Regular Grm (RG)

- Left Linear Grm (LLG)
- Right " (RLG)

RG : $S \rightarrow aA | Bb$
$A \rightarrow a$
$B \rightarrow b$

LLG : $S \rightarrow Aa | Bb$
$A \rightarrow a$
$B \rightarrow b$

RLG : $S \rightarrow aA | bB$
$A \rightarrow a$
$B \rightarrow b$

→ Inaccessible Token/Terminal.

$S \rightarrow aA$
$A \rightarrow a$
$B \rightarrow c$

c is inaccessible token/terminal

B is useless non terminal.

$R \rightarrow Th\ H\ T\ U$

$Th \rightarrow M\ Th \mid \epsilon$

$H \rightarrow CM \mid CD \mid DC \mid DCCC \mid C \mid CC \mid CCC \mid \epsilon$

$T \rightarrow XC \mid XL \mid LX \mid LXX \mid LXXX \mid X \mid XX \mid XXX \mid \epsilon$

$U \rightarrow IX \mid IV \mid VI \mid VII \mid VIII \mid II \mid III \mid \epsilon$

$R \rightarrow Th\ H\ T\ U$

$\rightarrow H\ T\ U$

$\rightarrow C\ T\ U$

$\rightarrow C\ XXX\ U$

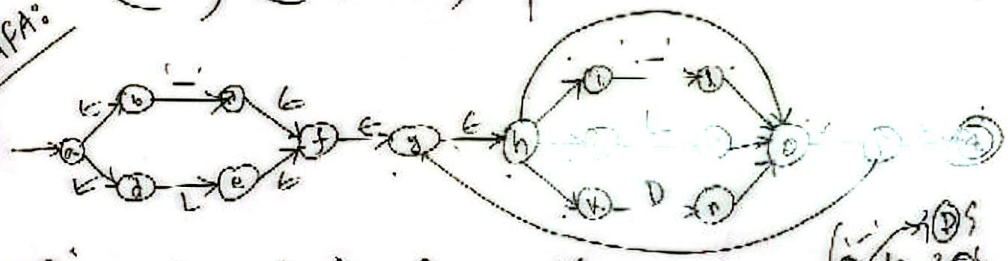
$\rightarrow C\ XXX\ VII$

I - II - III - IV - VI - VII - VIII - IX

RE:

$(_ | L)\,(_ | L | D)^*$ | $\Sigma = \{_, L, D\}$

ε-NFA:



DFA:

ε-closure (→a) = {→a, b, d} = A

{ε-closure(e)} = {e, f, g, h, i, j, k, o, p, q} = B

-A —_→ B

-A —L→ C

ε-closure(e) = {e, f, g, h, i, j, k, o, p, q} = C

{∅}

B —L→ D

{ε-closure(l)} = {g, h, i, j, k, l, o, p, q} = D

B —D→ E

{ε-closure(m)} = {g, h, i, j, k, m, o, p, q} = E

{ε-closure(n)} = {g, h, i, j, k, n, o, p, q} = F

C —_→ D

C —L→ E

C —D→ F

D —_→ D

D —L→ E

D —D→ F

E —_→ D

E —L→ E

E —D→ F

F —_→ D

F —L→ E

F —D→ F

State	_	L	D
→A	B	C	∅
B	D	E	F
C	D	E	F
D	D	E	F
E	D	E	F
F	D	E	F

State	_	L	D
→A	G	G	∅
G	H	H	H
H	G	G	G
K = {G, H}			

State	_	L	D
→A	K	K	∅
K	K	K	K

